

StyleMate — Consolidated Audit & Feature Plan

Merged from `bugs.md`, `second_feature.md`, `third_new_feature.md`, and `core_feature.md`.
176 total findings identified across all four files; 22 removed as duplicates; **145 distinct findings remain.**

Table of Contents

- 1. Bugs
 - 2. Features That Need Advancement
 - 3. Core Logic Reliability
 - 4. New Features
-

1. Bugs

1.1 Backend Bugs

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
BUG-01	Wardrobe router	<code>wardrobe.py</code> imports a nonexistent <code>WardrobeItem</code> model and undefined schemas. Router is dead code, never mounted in <code>main.py</code> .	Would immediately fail with <code>ImportError</code> if ever enabled; creates maintenance confusion.	Delete the dead router, or wire it to real models/schemas and mount it. Absorbs <code>second_feature.md</code> #38 (same dead code).	Critical
BUG-02	Calendar	<code>link_try_on_image()</code> auto-creates a <code>CalendarEntry</code> with hardcoded <code>user_id=1</code> when the entry doesn't exist.	Calendar data can be attributed to the wrong user.	Use the authenticated user's ID from the request context instead of a hardcoded value.	High
BUG-03	Weather	<code>rule_for_temp()</code> falls back to returning an integer instead of a rule dictionary when the temperature is outside known ranges.	Weather filtering can crash on invalid/extreme temperatures.	Return a proper rule dictionary on the fallback path.	High
BUG-04	Outfit explanations	Outfit explanation emits "balanced silhouette" twice, from two different scoring components.	Duplicate explanation fragments reduce explanation quality.	Deduplicate explanation fragments across scoring components.	High
BUG-05	Embeddings	Background embedding computation commits independently after the request completes; new items get no embedding until the background thread finishes. Also, the thread is lost on server restart. Absorbs <code>second_feature.md</code> S8.	Newly uploaded clothing temporarily receives neutral recommendation scores — users see bad recs until the job finishes.	Compute and store embeddings before the request returns, or use a durable queue with explicit completion handling.	High
BUG-06	Outfit scoring	Valid body types with empty rule sets	Certain body types never receive style optimization.	Give body types with no explicit rules a sensible default boost.	Medium

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		silently receive a zero boost.			
BUG-07	Feedback	Outfit feedback endpoint accepts arbitrary <code>user_id</code> values without ownership validation.	Training data can be polluted with fake feedback.	Validate that the requesting user owns the feedback before accepting it.	Medium
BUG-08	Tagging	Stores both <code>formality</code> and <code>formality_score</code> independently.	Values can drift and contradict each other.	Keep one source of truth; derive the other.	Medium
BUG-09	Caching	Delete/update routes redundantly invalidate cached item state after database cleanup.	Unnecessary work with no functional benefit.	Remove the redundant invalidation calls.	Medium
BUG-10	Tagging	Color sanity check defaults confidence to <code>1.0</code> when the confidence value is missing.	Real confidence-writing failures become hidden.	Treat missing confidence as unknown and surface it; never default to confident.	Medium
BUG-11	Capsule	Capsule builder queries wardrobe twice (in <code>pairing_engine.py</code> and <code>pair_cache.py</code>).	Extra database work on every capsule build.	Query once and reuse the result.	Medium
BUG-12	Packing	Packing endpoint blocks synchronously while waiting for Gemini (up to ~60 seconds). Absorbs <code>second_feature.md</code> S6 (same synchronous-blocking root).	Worker thread is blocked for the full Gemini response time; this will not scale.	Move Gemini calls to a background task/queue so the worker isn't blocked.	High
BUG-13	Embeddings	Missing embeddings silently return a neutral similarity score.	Difficult to diagnose missing embeddings.	Log or surface missing-embedding items explicitly.	Low
BUG-14	Fabric	Missing fabric information silently receives a neutral score.	Missing metadata isn't visible to anyone.	Surface missing fabric metadata instead of silently scoring neutral.	Low

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
BUG-15	Fit scoring	Unknown fit combinations default to a neutral score.	Unsupported fit combinations are hidden.	Expose unsupported fit combinations.	Low
BUG-16	Database	Database tables auto-create only for SQLite.	PostgreSQL deployments require manual schema setup.	Add migrations for PostgreSQL.	Low
BUG-17	Outfit scoring	Empty outfit returns 0.0 while a single-item outfit returns 0.5.	Inconsistent scoring baseline.	Make the scoring baseline consistent across outfit sizes.	Low
BUG-18	Color parsing	<code>_color_to_hsl()</code> doesn't normalize repeated whitespace.	Minor normalization inconsistency can cause lookup misses.	Normalize whitespace in the HSL parser.	Low
BUG-19	Config	Unknown <code>TAGGING_PROVIDER</code> values silently fall back to Gemini.	An environment typo can unexpectedly consume paid API credits.	Fail loudly on unknown provider values.	Low
BUG-20	Performance	Lazy loading triggers an additional SQL query during outfit explanation.	Minor N+1 performance issue.	Eager-load or batch the lazy query.	Low
BUG-21	Fashion rating	Rating cache key uses only the photo URL.	Different users sharing the same URL may receive cached results from each other.	Include user/context in the cache key.	Low
BUG-22	Upload	Retrying tagging re-uploads the same image.	Duplicate storage consumption.	Reuse the already-stored image on retry.	Low
BUG-23	Navigation	Settings icon calls <code>router.push("/app/settings")</code> but the correct route is <code>/settings</code> .	Tapping Settings shows "no route found"; navigation is broken.	Use the correct <code>/settings</code> route.	Medium
BUG-24	Capture	<code>.resize({ width: MAX_LONG_EDGE_PX })</code> always constrains width, not the actual long edge.	Portrait images get oversized (long edge unchecked); defeats the resize's purpose.	Compute the actual long edge and scale by it.	Medium
BUG-25	Wardrobe filters	Category & gender filter chips use <code>Set<String></code> allowing multi-selection	Users can accidentally select multiple categories/genders,	Make category/gender filters single-select.	Medium

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		instead of single-select.	contrary to expected behavior.		
BUG-26	Image editor	JPEG intermediate saves cause generational quality loss on every crop/rotate; only 90° rotation is offered; crop errors are silently dismissed with no user feedback.	Each crop/rotate degrades image quality; no fine rotation control; silent failures confuse users.	Preserve a lossless intermediate (original/PNG), add finer rotation control, and surface crop errors.	Medium
BUG-27	Capsule	Capsule default is 20 (<code>useState(20)</code>) but user wants 10.	Users get a larger-than-desired capsule by default.	Change the default to 10.	Medium
BUG-28	Add item	Adding items is slow — the upload → AI tagging → save roundtrip is slow, and embedding computation happens asynchronously after creation.	Poor UX; users wait too long after adding an item.	Speed up the roundtrip (optimistic UI, parallelize steps).	Medium
BUG-29	Navigation	Avatar press shows an Alert instead of navigating to a proper screen; no back button.	Users can't navigate to a profile/settings page with back support.	Navigate to a proper profile screen with a back button.	Low
BUG-30	Wardrobe grid	Odd item count leaves last row item without <code>columnWrapperStyle</code> spacing (<code>marginBottom</code>).	Last item in odd-count grid appears visually disconnected.	Apply consistent <code>columnWrapperStyle</code> spacing on the last row.	Low
BUG-31	Calendar	Outfit suggestions in the bottom sheet use a horizontal FlatList that doesn't work properly; should be a vertical scroll with 2 items per row.	Users can't browse outfit suggestions — horizontal scroll broken/unintuitive.	Use a vertical grid (2 per row) scroll.	Low
BUG-32	Outfit UI	Outfit-suggestions screen has UI gap issues; like/dislike button UX needs improvement.	Visual polish and interaction quality issues.	Polish spacing and the like/dislike interaction.	Low

1.2 Frontend Bugs

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
BUG-33	Calendar data model	The calendar data model stores only a single clothing item instead of a full outfit. Because of this, the Home "Upcoming" section shows only a date number (no outfit image) and the Calendar tab shows only dots/dates (no outfit images). Wear history and wear analytics are inaccurate.	Users can't see at a glance what outfit is scheduled or was worn; outfit history and analytics are inaccurate.	Extend the calendar model to store a complete outfit (top + bottom + footwear + accessories); render merged outfit images on Home "Upcoming" and on calendar dates. Aligns with the planned "Outfit Calendar & Planning" new feature.	High
BUG-34	Style match load	Initial fetch uses a small default <code>limit=6</code> per category and <code>PAGE_SIZE=5</code> ; "Load More" uses a higher limit that returns better, more diverse results.	Users see subpar static suggestions first — only getting good results after tapping "Load More"; the first load is either underwhelming or too dense.	Unify the initial and load-more fetch to use the same higher-quality query.	Medium
BUG-35	Style match sections	All category sections render regardless of selected item type or wardrobe count — e.g. "Matching Tops" shown even when the item is a top — and empty sections always render placeholder text instead of hiding.	Cluttered UI with irrelevant sections and pointless empty-state messages.	Render only relevant sections; hide empty sections entirely.	Medium
BUG-36	Wardrobe UI	Gap between wardrobe title and search bar; inconsistent colors throughout the screen (e.g. <code>filterBar</code> uses <code>colors.surface</code> while the search input uses hardcoded <code>"#f0f0f0"</code>). Absorbs <code>second_feature.md</code> #73 (same issue).	Looks disjointed — feels like separate components rather than a cohesive UI.	Unify spacing and use a single color token for related surfaces.	Medium

2. Features That Need Advancement

2.1 Weak / Underbaked Features Audit

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
ADV-01	Shopping	Meesho & Amazon providers fabricate fake placeholder products instead of returning real API results; the API also has no discriminator field to tell real from fake. Absorbs <code>second_feature.md</code> #56 (same root).	Users see blank images, ₹0 prices, and generic search redirects — destroying trust; the frontend can't render appropriately because it can't distinguish real from fake.	Return real product results (or clearly-flagged placeholders) and add a discriminator field.	High
ADV-02	Tagging	<code>style_tags</code> are computed by the AI but never saved — the frontend ignores them entirely.	Style-aware outfit recommendations can never work because the style data is lost before persistence.	Persist <code>style_tags</code> and surface them through the API and add-item flow.	High
ADV-03	Tagging	Missing "kurti" category in the frontend — the backend supports it but the frontend doesn't include it in the add-item form.	Indian ethnic wear becomes incorrectly categorized and unfilterable.	Add "kurti" to the frontend category options to match the backend taxonomy.	High
ADV-04	Shopping	Flipkart affiliate tracking is missing — product URLs aren't affiliate links.	No commission earned from purchases made through the app.	Generate affiliate-tracked URLs for Flipkart (and other providers).	High
ADV-05	Shopping	Only the first shopping provider is queried — the multi-provider architecture exists but is unused.	Backup providers never contribute results, so users get a smaller result pool.	Query providers in parallel/failover and merge results.	High
ADV-06	Outfit	Smart Outfit ignores extracted formality, season, and vibe — the NLP layer extracts them but the recommendation engine doesn't use them.	Results don't match the user's requested style.	Feed extracted formality/season/vibe into the outfit generator's filters.	High
ADV-07	Outfit	LightFM recommendation engine	Users think feedback is broken or ignored;	Wire feedback into LightFM retrain/predict	High

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		is effectively dead — like/dislike feedback is saved but never influences recommendations; score silently defaults to <code>0.5</code> when model is unavailable, masking the problem; buttons styled with very low alpha making them look disabled. Absorbs <code>second_feature.md</code> #74 and <code>bugs.md</code> H4.	there's no incentive to engage; app appears personalized when it isn't.	path; remove the silent <code>0.5</code> default (or make it visible); restyle like/dislike buttons so they look active.	
ADV-08	Shopping	FashionCLIP visual ranking downloads and embeds every product image per request — shop suggestions take 30–100 seconds; at scale, FashionCLIP on CPU (~2–5s per image) piles up under concurrent load. Absorbs <code>second_feature.md</code> S7 (same slow-inference root).	Users wait an extremely long time for shop suggestions; throughput collapses under concurrent load.	Cache embeddings, precompute/batch them, offload to GPU or a worker queue.	High
ADV-09	Shopping	Myntra & Ajio search URLs are malformed — query parameters are missing.	Links frequently open 404 pages.	Build correct query parameters for Myntra/Ajio URLs.	High
ADV-10	Tagging	Black denim is automatically changed to navy via a hardcoded override that increases confidence.	Incorrect color is stored with no review warning.	Remove the hardcoded override or flag it for review.	Medium
ADV-11	Tagging	Empty metadata can be saved — required AI review fields aren't validated before save.	Wardrobe contains incomplete items that hurt recommendations.	Validate required fields before allowing save.	Medium
ADV-12	Outfit	Explain Outfit endpoint lacks ownership checks — accepts arbitrary item IDs.	Possible wardrobe information leak.	Validate that the requesting user owns the item IDs.	Medium

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
ADV-13	Calendar	Duplicate calendar entries are allowed — no unique constraint on <code>(user_id, date)</code> .	Wear analytics become inflated.	Add a unique constraint on <code>(user_id, date)</code> .	Medium
ADV-14	Calendar	Deleting wardrobe items leaves broken calendar references — no cleanup of calendar records.	Dangling references and data loss.	Cascade-delete or clean up calendar references on item deletion.	Medium
ADV-15	Outfit	Weather endpoint exposes developer configuration message to users.	Poor UX and information leakage.	Return user-facing messages; keep setup details in logs.	Medium
ADV-16	Outfit	Gender filtering uses exact matching, which excludes unisex clothing.	Smaller recommendation pool.	Treat unisex items as eligible for either gender filter.	Medium
ADV-17	Style Match	<code>/style-advice</code> endpoint exists but UI never calls it — fully implemented backend feature is orphaned. Users never access the most advanced shopping assistant. Absorbs <code>second_feature.md</code> #20 (Complete Outfit endpoint never used) and #64 (cached outfit generation never used — same root: built but never wired to UI).	Several advanced backend capabilities are built but never connected to the user interface; the frontend uses a simpler endpoint and never benefits from outfit caching or style advice.	Wire these endpoints into the UI (or remove them); use the cache to keep outfit diversity across pages.	Medium
ADV-18	Tagging	Formality review indicator is never shown — review flag always disabled.	Incorrect formality scores remain unnoticed.	Show the review flag in the add-item UI.	Medium
ADV-19	Tagging	Formality score can be stored as null.	Outfit filtering becomes inconsistent.	Validate non-null formality before save (or define null semantics).	Medium
ADV-20	Shopping	Complete the Look uses generic search URLs instead of actual product links.	Poor shopping experience and no affiliate revenue.	Link to specific products, not generic searches.	Medium

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
ADV-21	Shopping	Duplicate Meesho links generated — same store appears twice.	Confusing UI.	Deduplicate shop links in the response.	Medium
ADV-22	Outfit	Fallback "Why this works?" explanation shows internal compatibility labels.	AI explanation appears broken to users.	Write human-readable fallback explanations.	Medium
ADV-23	Tagging	Review status isn't persisted — AI uncertainty disappears after save.	Users can't revisit uncertain classifications.	Persist review status with the item.	Medium
ADV-24	Tagging	Saree/Lehenga categorized as Kurti — incorrect garment hierarchy.	Outfit generation becomes inaccurate.	Correct the taxonomy hierarchy so sarees/lehengas map to their own categories.	Medium
ADV-25	Shopping	Gap analysis only counts categories — doesn't recognize similar colors or styles; always falls back to "beige" when gap color can't be determined. Absorbs <code>bugs.md</code> M1 (same color-blind gap-analysis root).	Suggests purchases the user effectively already owns; repetitive, tone-deaf suggestions erode trust.	Make gap detection color- and style-aware; pick a gap color from the wardrobe palette instead of a hardcoded fallback.	Medium
ADV-26	Outfit	"Date" interpreted as "Party" by the NLP keyword mapping.	Wrong outfit recommendations for casual dates.	Disambiguate "date" in the NLP keyword mapping.	Medium
ADV-27	Outfit	Outfit diversity resets between pages — Load More ignores previous diversity state.	Repetitive outfit suggestions across pages.	Carry diversity state across pages (use the cache).	Medium
ADV-28	Outfit	Neutral wardrobes generate repetitive outfits because neutral colors receive identical scores.	Low outfit variety.	Add tie-breaking / diversity logic for equal-scoring neutral items.	Medium
ADV-29	Outfit	Small wardrobes fail silently — no onboarding guidance when wardrobe is too small to generate outfits.	New users receive empty recommendations.	Add guidance / onboarding for small wardrobes.	Medium

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
ADV-30	Shopping	Missing affiliate disclosure for affiliate links.	Regulatory compliance risk.	Add a visible affiliate disclosure.	Medium
ADV-31	Style Match	Shopping recommendations use static templates with personalized scores applied to fake products.	Personalization appears misleading.	Apply personalized scores to real products.	Medium
ADV-32	Capsule	Outfit count measures pairs, not complete outfits.	Misleading statistics — users overestimate item usefulness.	Report complete-outfit counts instead of pair counts.	Medium
ADV-33	Tagging	Duplicate FORMALITY_MAP definitions in frontend and backend.	Two copies may diverge — maintenance risk.	Single-source the FORMALITY_MAP in one place.	Medium
ADV-34	Tagging	Gemini lacks a dedicated gender classifier — relies entirely on free-form LLM output.	Less reliable gender classification than a dedicated classifier.	Add a dedicated gender classification step (or the 3-phrase classifier path).	Medium
ADV-35	Tagging	Confidence threshold too low — near-random predictions are accepted.	Incorrect classifications stored confidently.	Raise the confidence threshold (align with the review threshold).	Medium
ADV-36	Tagging	Pattern list mismatch — frontend includes "other"; backend doesn't.	Non-standard values get stored.	Align the pattern lists between frontend and backend.	Medium
ADV-37	Shopping	Shopping endpoints handle failures inconsistently — different APIs return different error behavior; any provider exception becomes HTTP 502 instead of degrading gracefully. Absorbs <code>s.md</code> <code>bug</code> M11 (same inconsistent error-handling root).	Temporary provider failures break the whole endpoint; inconsistent user experience.	Standardize error handling and degrade gracefully (partial results) on provider failures.	Medium
ADV-38	Shopping	Shop link queries are too generic — missing	Low-quality shopping results.	Pass gender/occasion/subtype into shop queries.	Medium

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		gender, occasion, and subtype.			
ADV-39	Style Match	Duplicate wardrobe lists in API response — same data serialized twice.	Wasted bandwidth.	Serialize the wardrobe list once.	Medium
ADV-40	Tagging	Dupatta missing from Gemini taxonomy — Gemini never predicts it correctly.	Manual correction always required.	Add dupatta to the Gemini taxonomy.	Medium
ADV-41	Tagging	Unknown occasions default to formality 3 — an invalid fallback value.	Formal clothing misclassified.	Choose a correct fallback or require a value.	Medium
ADV-42	Tagging	No visible "Review Required" banner in the add-item flow.	Review state easy to miss; users unknowingly save incomplete data.	Show a visible review-required banner.	Medium
ADV-43	Tagging	No retry for FashionCLIP failures — partial failures leave missing metadata.	Less reliable tagging.	Add retry/backoff for FashionCLIP calls.	Low
ADV-44	Tagging	Color validation only checks white — other color mistakes are ignored.	Incorrect colors persist.	Validate the full set of colors.	Low
ADV-45	Tagging	No content-type validation when downloading images — HTML pages can be processed as images.	Invalid AI tagging results.	Validate content-type before processing downloads.	Low
ADV-46	Outfit	Weather recommendations silently degrade — falls back without informing users.	Reduced trust in recommendations.	Surface weather fallbacks to the user.	Low
ADV-47	Outfit	Fragile SQL LIKE occasion filtering — uses substring matching.	Potential future filtering bugs.	Use exact/tagged matching instead of substring LIKE.	Low
ADV-48	Outfit	Duplicate feedback records possible — no deduplication.	Recommendation dataset becomes noisy.	Deduplicate feedback records.	Low

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
ADV-49	Shopping	Meesho URL builders inconsistent — two different builders produce different URLs.	Confusing behavior.	Unify into one URL builder.	Low
ADV-50	Shopping	Shop match fit metadata copied from source item instead of the recommended product.	Misleading compatibility.	Use the recommended product's own fit metadata.	Low
ADV-51	Shopping	Provider configuration silently ignores typos — unknown providers are skipped.	Admins believe providers are active when they aren't.	Fail loudly on unknown provider names.	Low
ADV-52	Style Match	Owned and generated items mixed together in results.	Difficult to distinguish owned vs. suggested — confusing shopping experience.	Clearly separate and label owned vs. generated items.	Low
ADV-53	Style Match	Fashion Rating unavailable without Gemini — no heuristic fallback.	Feature unusable offline / in free mode.	Add a heuristic fallback rating path.	Low
ADV-54	Style Match	Packing ignores real weather service — hallucinates weather instead of using the API.	Packing recommendations may be wrong.	Use the real weather service for packing input.	Low
ADV-55	Calendar	Same outfit can be scheduled repeatedly — no duplicate-outfit validation.	Poor planning experience.	Warn / reject scheduling the same outfit repeatedly.	Low
ADV-56	Tagging	Accessories taxonomy too broad — many culturally distinct accessories grouped together.	Reduced classification accuracy.	Split accessories into more specific categories.	Low
ADV-57	Style Match	Packing list capped regardless of trip duration — 14-day and 60-day trips look similar.	Long-trip recommendations become unrealistic.	Scale the packing list with trip length.	Low
ADV-58	Style Match	Packing shopping links ignore gender — generic search queries.	Less relevant products.	Include gender in packing shop queries.	Low

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
ADV-59	Style Match	Fashion rating ignores wardrobe context — rates only the uploaded image.	Doesn't reflect personal style.	Include wardrobe context in the rating.	Low
ADV-60	Style Match	Packing destination unsanitized — prompt injection possibility.	Security and reliability risk.	Sanitize / validate destination input before prompting the model.	Low
ADV-61	Style Match	Non-neutral suggestions bypass the 70% quality threshold — <code>STYLE_VARIETY_THRESHOLD=62</code> lets colorful items through at 62% instead of the standard 70%.	Users see low-match suggestions (62–69%) that don't meet the quality bar.	Raise the variety threshold to match the standard, or justify the lower bar.	Low
ADV-62	Wardrobe	Filter chips are bulky, lack visual hierarchy and multi-select clarity — large flat chip rows take too much space with no grouping, icons, or active-state contrast.	Filter bar feels oversized and cluttered; harder to browse wardrobe.	Restructure filter chips with grouping, icons, and clearer active states.	Low

Summary by Feature (corrected from source — computed from actual audit rows):

Feature	Findings
Tagging	19
Shopping	14
Outfit Suggestions	13
Style Match	11
Calendar	3
Wardrobe	1
Capsule Wardrobe	1

2.2 Top 5 Systemic Risks

- 1. Fake Shopping Infrastructure** — Shopping providers fabricate products instead of retrieving real listings; downstream shopping features inherit misleading data.
- 2. Dead Recommendation Pipeline** — LightFM personalization, cached outfits, and `/style-advice` are built but never actually used.

3. **Calendar Design Limitation** — Calendar stores only a single clothing item instead of a complete outfit, degrading outfit history and analytics.
 4. **Tagging Pipeline Loses Valuable AI Output** — `style_tags` are generated but discarded before persistence, preventing style-aware recommendations.
 5. **Disconnected Features** — Several advanced backend capabilities (Style Advice, Complete Outfit, Cached Outfits) are implemented but never connected to the user interface.
-

2.3 Scalability Issues

These work now but will break or degrade with more users.

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
SCL-01	Database	SQLite single-writer bottleneck — <code>check_same_thread=False</code> allows one concurrent write at a time.	With 5+ users, writes queue and time out.	Migrate to a database that supports concurrent writes (e.g., PostgreSQL).	Critical
SCL-02	Auth	No auth system — hardcoded <code>DEMO_USER_ID=1</code> everywhere. No JWT, sessions, or multi-user support.	Cannot go live without auth; all user data is shared.	Implement the authentication plan in Section 3.3 (this is a prerequisite for launch).	Critical
SCL-03	Background jobs	Thread-based background jobs (<code>Thread(target=...).start()</code>) for embeddings and photo cleanup, instead of a proper queue.	Threads don't scale across processes and are lost on restart.	Move background work to a durable job queue.	High
SCL-04	API	No pagination on wardrobe/items API — <code>GET /clothing/</code> fetches ALL items.	With 500+ items, response size and query time grow linearly.	Add pagination / limit-offset to list endpoints.	High
SCL-05	Caching	In-memory caches (TTLCache in weather, try-on, shopping services) don't work across instances.	Need Redis for multi-instance deployment.	Replace in-memory caches with a shared cache (e.g., Redis).	High
SCL-06	Security	CORS wide open — <code>allow_origins=["*"]</code> .	Acceptable for dev, but security risk in production.	Restrict allowed origins to the app's real domains.	Medium
SCL-07	Security	No API rate limiting — any endpoint can be hammered; only try-on has per-user limiting.	Abuse / DoS risk.	Add rate limiting globally.	Medium
SCL-08	Upload	No client-side file size check before upload — server rejects >10 MB but upload already consumed bandwidth.	Users wait, then get an error.	Validate file size on the client before uploading.	Low
SCL-09	Client	No request timeout handling on client — <code>api.ts</code> has no timeout.	A hanging server means the app hangs forever.	Add timeouts / abort handling to API calls.	Medium

3. Core Logic Reliability

Three kinds of fix are mixed in this section — they need different solutions:

-  **Model / coverage fix** — a genuinely fixable accuracy ceiling (better candidates, more categories)
-  **Not visually solvable** — no AI model can determine this from a photo alone; needs a different data source entirely (user input, or stop scoring it)
-  **Pure code bug** — no AI involved, just a logic error; cheapest to fix

Plain-language note: Throughout this section, the "match %" shown to users is a weighted blend of 9 scoring signals. When we say a signal is "near-random," it means the AI is essentially guessing — so the final number overstates how well two items actually go together. This matters because users trust that number to make outfit decisions.

Headline Finding

23% of the total match-score weight (fabric 8% + season 5% + style tag 6% + embellishment 4%) relies on fields the source analysis found to be near-random or worse from FashionCLIP. The 3 most reliable signals (embedding 22% + fit 7% + silhouette 6%) only add up to 35% of the total. The "match %" shown to users is currently a blend where nearly a quarter of the formula is closer to noise than signal.

3.1 Tagging Fields with Direct Zero-Shot Classification

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
CORE-01	Tagging (category)	None — this field works reliably (7 options). Verified.	N/A — no issue.	No action needed.	—
CORE-02	Tagging (pattern)	Candidate list only has 4 options; floral, plaid, polka dot, geometric, and tie-dye are missing — those patterns get forced into "printed" or "solid."	Users can't filter or match these common patterns correctly, degrading pairing quality.	Expand the pattern candidate list 🔧 (coverage fix).	Low
CORE-03	Tagging (dominant_color)	FashionCLIP predicts from only 13 colors while the scoring table supports ~50; burgundy, teal, olive, sage, lavender, peach, coral, mint, turquoise, maroon all collapse to a generic parent (e.g., "red" for burgundy, "green" for olive). Plain language: The AI only "sees" 13 colors while the scoring engine understands ~50, so many distinct colors get treated as identical before scoring even happens.	Distinct colors are scored as identical, so matching and shopping queries are wrong for many items.	Expand FashionCLIP's color candidates or switch to pixel-based extraction 🔧.	Medium
CORE-04	Tagging (occasion_tag)	Not visually determinable — a black blazer could be "office" or "party" with equal validity. Confidence sits near noise (~0.12–0.15), below the review threshold, so almost everything needs review but isn't flagged. Plain language: No AI can reliably guess an occasion from a photo — the field is basically a coin flip, yet it's presented as confident output.	Occasion-driven filtering and hard-rule scoring inherit near-random input.	Make occasion user-set at upload, or infer from category + formality instead 🚫 (not visually solvable — needs a product decision, not a better model).	High
CORE-05	Tagging (season)	Weak visual signal — a cotton t-shirt has no real season; only extreme cases (heavy coat = winter) are reliable. Plain language:	Two items both defaulting to a guessed "summer" can score as a	Reduce its scoring weight, or let users set season 🚫	High

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		The photo rarely contains enough information to say what season an item is for.	confident match when they shouldn't.	(mostly not visually solvable).	
CORE-06	Tagging (fabric_type)	Fundamentally not visually determinable — even a human can't reliably tell cotton vs. linen vs. synthetic from a photo; denim is the one exception. Plain language: The AI is guessing at a property that mostly can't be seen in a picture.	Fabric contributes 8% of the match score and the hardcoded fabric rules encode subjective opinions as fact.	Make fabric user-set at upload, or stop scoring it  (not visually solvable — product decision).	High
CORE-07	Tagging (fit_type)	Depends on garment draping — a loose item photographed flat may look "regular." Only 4 fit types and 16 combinations exist — "relaxed," "tapered," "skinny" aren't represented.	Fit misclassification feeds the 7%-weight fit signal; common fits are invisible.	Partially fixable with better photo guidance at upload  ; coverage gap, not a reliability issue.	Medium
CORE-08	Tagging (sleeve_length)	None — clear visual cue, this one works. Verified.	N/A — no issue.	No action needed.	—
CORE-09	Tagging (subcategory)	Two-stage zero-shot (group → specific label); subtle visual distinctions like "skinny" vs. "straight_leg" or "palazzo" vs. "trousers" are genuinely hard for the model. Plain language: Telling apart very similar pant styles from a photo is beyond what current AI can reliably do.	Subcategory errors cascade into pairing and filtering inaccuracies.	Model/coverage fix — realistic ceiling, not a bug  .	Low
CORE-10	Tagging (garment_length)	None — length from hemline is a real visual cue; working as intended.	N/A — no issue.	No action needed.	—
CORE-11	Tagging (embellishments)	9 binary checks at threshold 0.15 — barely above noise; "buttons" fires on every button-up shirt; seams/zippers/pockets trigger false positives. Plain language: The confidence bar is set so low	Wrong embellishment flags feed the 4%-weight embellishment signal and confuse users.	Raise the threshold — pure config fix  .	High

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		that almost any garment with buttons or pockets gets flagged, even when there's nothing special about it.			
CORE-12	Tagging (style_tags)	Multi-label at threshold 0.12 — extremely low; a plain t-shirt will likely match "fitted" or "structured" at 0.12 purely by chance. Plain language: The confidence bar is so low that random, meaningless tags get stored as real style attributes.	Noise leaks into the 6%-weight style-tag signal, incorrectly triggering body-type boosts.	Raise the threshold — pure config fix 🦋.	High
CORE-13	Tagging (target_gender)	Dedicated 3-pharse classifier, margin 0.05; FashionCLIP has no real concept of gendered clothing — results are close to random and the margin is too small to mean anything even if they weren't. Plain language: The AI is effectively guessing an item's gender, and a tiny similarity gap is treated as a confident answer.	Unisex clothing is frequently misclassified, shrinking the recommendation pool and mis-filtering items.	Make gender user-set or treat it as a soft filter 🚫 (not visually solvable — product decision).	High
CORE-14	Tagging (brand)	Only populated via Gemini — stays permanently NULL with the default free path (<code>TAGGING_PROVIDER=fashion_clip</code>).	Brand can't be used for matching or filtering in the default mode.	Needs user input 🚫 (not visually solvable for most items).	High
CORE-15	Tagging (formality_score)	Derived from <code>occasion_tag</code> , compounding T4's unreliability — an already-shaky value built on another shaky value. Plain language: If the occasion guess is random (which it is), the formality score derived from it is also random.	Formality filtering becomes inconsistent wherever the underlying occasion guess is wrong.	Fixing T4 (making occasion user-set) fixes this too 🦋 + 🚫 combined.	High

3.2 Tagging Fields Handled Outside Standard Candidate-List

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
CORE-16	Tagging (subcategory method)	Two-stage zero-shot — subtle distinctions ("skinny" vs. "straight_leg", "palazzo" vs. "trousers") genuinely hard for the model.	Subcategory errors cascade into pairing inaccuracies.	Model/coverage fix  (same root as CORE-09, kept here for completeness).	Low
CORE-17	Tagging (garment_length method)	Single zero-shot, 7 options — reasonable; length from hemline is a real visual cue.	N/A — working as intended.	No action needed.	—

3.3 Color Matching Structural Issues

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
CORE-18	Color matching	FashionCLIP predicts from only 13 colors; the scoring table (<code>HSL MAP</code>) supports ~50. "Burgundy," "maroon," "crimson," and "cherry" all collapse to the same generic "red" before scoring ever happens. Absorbs the "olive green vs. olive" mismatch (a specific instance of the mapping gap). Also absorbs the "unknown colors fall back to static recommendations" issue (unmapped colors return <code>None</code> → every pairing silently scores 0.5). Plain language: The AI only "sees" 13 colors while the scoring engine understands ~50 — so many distinct colors get lumped together, and colors it doesn't recognize at all are treated as matching everything moderately well.	Distinct colors are scored identically; unmapped items seem to match everything; shopping queries go generic.	Expand FashionCLIP's color candidates, or switch to pixel-based extraction 🔑 (resolves both the mapping mismatch and the unmapped-color fallback).	Medium
CORE-19	Color matching	Fuzzy substring matching is order-dependent — <code>"red" in "wine red"</code> matches "red" before "wine" is ever checked, because dictionary order decides the winner. Plain language: Which color name "wins" is decided by the order the code was written, not which color is actually closer.	Derived color names silently collapse to their generic parent, losing the nuance the color table was built for.	Score against all candidates and pick the best match 🏆 (pure code bug).	High
CORE-20	Color matching	Neutral detection is oversensitive — any color with saturation under 15 or lightness outside 10–90 is treated as neutral. Plain language: A clearly colorful pastel blue can be classified as "neutral," which means it's treated as safe to pair with almost anything — even colors it would actually clash with.	A genuinely colorful pastel can score 0.9 with everything, including colors it clashes with.	Tune the neutral thresholds 🏆 (pure code bug).	High
CORE-21	Color matching	Clash list is too small — only 7 hardcoded clashing color pairs	Real, well-known clashes	Needs real fashion-rule input, not just	Low

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		exist.	(navy+black, brown+black, gold+silver, red+pink, blue+purple) are never penalized, inflating scores for genuinely bad pairings.	code  (content/coverage fix).	

3.4 Scoring Signal Reliability (the 9 weighted components)

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
CORE-22	Scoring (Color 30%)	Carries all CORE-18–21 issues — oversensitive neutral detection scores real clashes as 0.9; navy+black never penalized (not in the clash list). Plain language: The biggest piece of the match % is partly built on noisy or incomplete color data.	The largest single component of the match score is unreliable for many items.	Fix the color-matching issues above (CORE-18–21), then revisit this weight 🛠️.	Medium
CORE-23	Scoring (Embedding 22%)	Two visually near-identical items (e.g., two white t-shirts) score as a near-perfect "match" despite being the same item type; genuinely good pairings with visually different embeddings (silk blouse + wool trousers) can score low. Plain language: The AI scores how similar two items <i>look</i> , not whether they're a good outfit together — by design.	The second-biggest weight rewards same-looking items and punishes visually-different-but-good pairings.	Structural — blend with non-visual signals; consider penalizing same-category pairs.	Medium
CORE-24	Scoring (Hard rules 12%)	Two items both (mis)tagged <code>occasion_</code> <code>tag="casual"</code> score a perfect match — even if the tag itself was a coin flip.	Directly inherits CORE-04's unreliability into a confident score.	Fix CORE-04 (occasion); reduce rule weight until then.	Medium
CORE-25	Scoring (Fabric 8%)	A hardcoded "good" pair (e.g., denim+leather) scores well even when it looks bad — the table encodes one opinion as fact. A real clash (silk+denim) scores low even when it can look great — same issue, opposite direction.	Inherited from CORE-06's unreliability, compounded by a subjective hardcoded table.	Reduce weight or make fabric user-set (per CORE-06).	Medium

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
CORE-26	Scoring (Season 5%)	Two items both defaulting to "summer" (a common fallback guess) score as a confident match.	Inherited from CORE-05's unreliability.	Fix CORE-05 (reduce weight / user-set); revisit weight.	Medium
CORE-27	Scoring (Style tag 6%)	The 0.12 threshold (CORE-12) lets noise through, incorrectly triggering body-type boosts; most items never clear 0.12 at all, so the signal mostly sits at neutral.	Most items contribute nothing to this signal; the ones that do may be noise-driven.	Fix CORE-12 (raise threshold).	Medium
CORE-28	Scoring (Silhouette 6%)	Only two possible outputs (0.9 or 0.5) — a slightly-off pairing scores identically to a completely wrong one; only covers top+bottom — an A-line dress + fitted jacket has no rule at all and defaults to 0.5. Plain language: The silhouette scorer is binary and only knows about tops and bottoms — any other garment combination gets the same middle-of-the-road score.	Genuinely wrong silhouette pairings get the same score as merely "off" ones; dresses and ethnic wear are invisible.	Extend silhouette rules to cover dresses, ethnic wear, and outerwear layering 🛠️ (coverage fix). Also absorbs <code>bug s.md</code> H6 (same ethnic-wear gap) and <code>second_feature.md</code> #17 (same dresses+outerwear generation omission).	High
CORE-29	Scoring (Embellishment 4%)	Never actually penalizes — the floor is 0.50, so this signal can only help, never hurt, even when it should. Plain language: Even when embellishments clash, the worst this signal does is nothing — it never pulls the score down.	Design choice, not a bug — but worth deciding if that's actually intended.	Decide if the floor is intended; if not, allow negative impact 🛠️.	Medium

3.5 Outfit-Level Bonus / Penalty Edge Cases

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
CORE-30	Outfit scoring	Complementary + analogous bonuses can both fire — +0.10 and +0.12 are separate additive bonuses that stack to +0.22 on the same outfit. Plain language: The outfit can get double-credited for two overlapping definitions of "the colors work," inflating the score beyond what either bonus alone intended.	Score inflation for certain color combinations.	Ensure only one color harmony bonus applies per outfit 🐛 (pure code bug).	High
CORE-31	Outfit scoring	Bonus is flat, not proportional to outfit size — a 2-item outfit and a 5-item outfit get the exact same flat bonus. Plain language: Smaller outfits get a disproportionately bigger boost relative to their size.	Smaller outfits are inflated relative to larger ones.	Make the bonus proportional to outfit size 🐛 (pure code bug).	High
CORE-32	Outfit scoring	Busy-pattern penalty is uniform — any 2+ busy patterns trigger the same flat 50% score cut, regardless of which patterns.	Striped+floral (a real clash) gets treated identically to plaid+checked (which might work).	Needs real fashion-rule nuance, not just a formula fix 🔧.	Low
CORE-33	Outfit scoring	All-neutral palette is detected but never scored — the code identifies an all-neutral outfit and adds a note to the explanation text, but the score itself is unaffected. Plain language: The detection work is real but currently has zero functional impact — it's dead code.	The detection is wired up but doesn't actually change the score.	Either wire it into scoring or remove the dead detection 🐛 (pure code bug).	High

Suggested Fix Order

1. **Fix the pure code bugs first** (CORE-19, CORE-20, CORE-30, CORE-31, CORE-33) — no AI tradeoffs, no design decisions, just correcting logic that's already wrong on its own terms. Cheapest, safest, highest confidence.
2. **Decide the "not visually solvable" fields** (CORE-04 occasion, CORE-06 fabric, CORE-13 gender) — these need a product decision, not more engineering: either make them user-set at upload (a quick form field), or stop presenting them as confident AI output and reduce their scoring weight accordingly. Trying to fix these with a "better model" will not work.
3. **Apply the pixel-based color extraction** (CORE-18) — this single fix resolves the color-count mismatch and significantly reduces the color signal's false-positive risk at the same time.

4. **Rebalance the 9 scoring weights** (CORE-22–29) — right now 23% of the formula leans on fields being actively improved or reduced; revisit the weight split after those changes land rather than before.
 5. **Coverage gaps** (CORE-02, CORE-09, CORE-21, CORE-07, CORE-28) — real but lower urgency; these are "the feature works but isn't comprehensive yet" issues.
-

4. New Features

4.1 Core New Features

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
NEW-01	Virtual Try-On	Not implemented — no way to see a full outfit (top + bottom + footwear + accessories) on a person as one image; current try-on only handles single items.	Users can't visualize an entire outfit before purchase — improving engagement and confidence in styling decisions.	Research and integrate an open-source virtual try-on model that generates a single realistic image of the person wearing the complete outfit.	High
NEW-02	Outfit Completion	Not implemented — when a user selects two or more items (e.g. top + bottom), there are no suggestions for the remaining complementary pieces.	Users can build outfits piece by piece and get intelligent suggestions for what's missing — closing the gap between partial selection and a full look.	Build a completion engine that suggests the missing complementary items (footwear, accessories, layering).	High
NEW-03	Family Wardrobe	Not implemented — family members can't connect wardrobes or gift each other items.	Data-driven gifting — instead of guessing sizes and style, users get recommendations for what their family member would actually wear. Increases shopping engagement and affiliate revenue.	Add family / connected wardrobes and style-gap-based gifting suggestions.	Medium
NEW-04	Style This Item	Not implemented — no way to get multiple outfit ideas built around 1–2 selected items.	Shows how many ways a single item can be worn — increases perceived wardrobe value, encourages experimentation, and drives shopping suggestions for missing pieces.	Generate 4–6 complete outfit variations per base item(s), each with match percentage and a reason.	High

4.2 Calendar New Features

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
NEW-05	Outfit Calendar & Planning	Not implemented — the calendar only shows dots/dates; it can't show outfit images, plan ahead, or lock finalized outfits.	Visual outfit history, daily tracking, outfit planning, and protection of finalized outfits; enhances virtual try-on quality with full-outfit generation.	Replace the calendar UI with an outfit-based calendar (image per date, planned outfits, outfit lock); integrate the full-outfit virtual try-on model (NEW-01).	High
NEW-06	Today's Outfit Widget	Not implemented — the Home page has no "Today's Outfit" card.	Gives users instant access to the current day's outfit, improves Home page engagement, and provides quick navigation to today's look without opening the Outfit Calendar.	Add a Today's Outfit widget on Home that opens that day's outfit details.	Medium

4.3 Authentication & Authorization (Missing — PREREQUISITE FOR LAUNCH)

Context: The entire codebase hardcodes `DEMO_USER_ID=1` — no login, no register, no session management. The app cannot go live without this. See also SCL-02 (Scalability).

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
NEW-07	Register	Not implemented — no way to create an account.	Users can't create accounts — first step toward personalization and multi-user support.	<code>POST /auth/register</code> — validate email, hash password with bcrypt (cost=12), store in <code>users</code> table, return <code>{ access_token, refresh_token }</code> . Store refresh token hash in DB for rotation. Frontend: register screen, store tokens in <code>expo-secure-store</code> , navigate to home. Validation: strong password (min 8 chars, uppercase + number), email format, prevent duplicate emails.	Critical
NEW-08	Login	Not implemented — no login.	Returning users can access their wardrobe — session management replaces hardcoded IDs.	<code>POST /auth/login</code> — verify with bcrypt, issue access token (15 min) + refresh token (7 days) with <code>user_id</code> and <code>role</code> in payload. Rate-limit to 5 attempts/15 min per email. Frontend: store tokens in <code>expo-secure-store</code> , decode JWT to get <code>user_id</code> . Never store raw tokens in <code>AsyncStorage</code> — use <code>expo-secure-store</code> (keychain).	Critical
NEW-09	Token Refresh	Not implemented — no token refresh mechanism.	Keeps users logged in without repeated login prompts.	<code>POST /auth/refresh</code> — accept refresh token, verify signature + expiry, check hash against DB, issue new pair, invalidate old refresh token (rotation prevents replay attacks). Frontend: Axios/fetch interceptor in <code>api.ts</code> — on 401, attempt refresh; if refresh fails, redirect to login; queue pending requests during refresh.	Critical
NEW-10	Logout	Not implemented — no logout.	Users can securely end sessions — important for shared devices.	<code>POST /auth/logout</code> — delete refresh token from DB; optionally maintain a Redis blacklist of access tokens until natural expiry. Frontend: clear tokens from secure store, navigate to login screen.	Critical
NEW-11	Auth Middleware	Not implemented — no JWT verification on	Replaces all hardcoded user IDs — every	FastAPI <code>Depends(get_current_user)</code> middleware that decodes JWT, checks expiry,	Critical

ID	Area	What's Wrong	Why It Matters	Fix Direction	Priority
		protected routes; <code>DEMO_USER_ID=1</code> hardcoded everywhere.	endpoint gets the real authenticated user.	attaches <code>user_id</code> to request. Apply globally with opt-out for public routes (<code>/auth/*</code> , <code>/docs</code>). Remove all hardcoded <code>user_id=1</code> across routers. Optionally add scoped roles (<code>admin</code> , <code>user</code>) for future admin features.	
NEW-12	Protected Navigation	Not implemented — screens aren't guarded by auth state.	Prevents accessing the app without login — essential once auth is enforced on the backend.	Wrap navigation in an auth context. Use <code>expo-secure-store</code> to check for token on app launch. If no token → show <code>AuthNavigator</code> (Login/Register stack). If token exists → show <code>AppNavigator</code> (tabs + screens). Handle token expiry gracefully — if refresh fails, clear tokens and redirect to login.	Critical
NEW-13	Password Reset	Not implemented — no password recovery.	Users who forget passwords can recover accounts — reduces support burden.	<code>POST /auth/forgot-password</code> — generate reset token (random 32 bytes), hash and store in DB with 1 hr TTL, send email via SendGrid/SES. <code>POST /auth/reset-password</code> — verify reset token, update password hash. Frontend: forgot password screen → email input → success message. Reset screen (from email link) → new password + confirm.	High
NEW-14	Social Login	Not implemented — no Google/Apple sign-in.	Frictionless sign-up — reduces drop-off at registration. Critical for mobile.	Library: <code>expo-auth-session</code> + <code>@react-native-google-signin/google-signin</code> for Google, <code>expo-apple-authentication</code> for Apple. Backend: <code>POST /auth/social</code> — accept <code>{ provider, id_token }</code> , verify with Google/Apple public keys, find-or-create user by <code>(provider, provider_id)</code> , issue JWT pair. Apple sign-in is mandatory for App Store review if using social login.	High